

PYTHON · OBJECT-ORIENTED PROGRAMMING

OOP Practice Workbook

Nine Class-Design Problems Across Real-World Domains

Each problem includes a scenario, a class to design, method specifications, sample data with expected output, constraints, and the concepts practised.

(Questions only — no solution code included.)

Questions at a Glance

The problems share a similar shape — build a class, store data in the right structure, then write methods that generate reports. They are presented below in a clean sequential order.

#	System	Domain	Core Data Structure
1	Employee Attendance System	Human Resources	Dict of lists
2	Inventory Management System	Retail / Inventory	Nested dict
3	Hospital Appointment System	Healthcare	List of tuples
4	Digital Wallet System	FinTech	Dict + dict of lists
5	Online Shopping Cart System	E-Commerce	Nested dict
6	Movie Streaming Platform	Entertainment	Two dicts of lists
7	Gaming Tournament System	Gaming	Dict of lists
8	University Course Feedback	Education	Dict of lists
9	Loan Management System	FinTech	Dict of lists

How to read each question

For every method you get its signature, its parameters, a plain-English description of what it must do, and a sample input/output. Sample outputs show the result your method should produce — they are part of the specification, not a worked solution.

Question 1 — Employee Attendance System

Domain · Human Resources

Problem Statement

A company employs several workers across different departments. On every working day each employee is marked either Present ("P") or Absent ("A").

The HR department wants to analyse these attendance records to find employees with perfect attendance, count the absences of individual employees, calculate attendance percentages, and identify the employee with the highest attendance percentage.

At the moment the records are kept manually, which makes reporting slow and error-prone. The company therefore needs a simple attendance-management system.

Objective

Design and implement a Python class named `EmployeeAttendanceSystem` that stores employee attendance records and generates attendance reports. Use suitable data structures such as dictionaries and lists.

Class Structure

CLASS NAME

```
class EmployeeAttendanceSystem
```

PUBLIC ATTRIBUTE

`attendance` — dictionary

Stores each employee's attendance. The key is the employee name (string) and the value is a list of statuses, where each status is "P" (present) or "A" (absent). It must be created inside the constructor.

```
{
    "Ravi": ["P", "P", "A"],
    "Meera": ["P", "P", "P"]
}
```

Methods

Method 1 · `__init__`

SIGNATURE

```
def __init__(self, attendance=None)
```

PARAMETERS

Parameter	Type	Description
attendance	dict (optional)	Existing attendance records. Defaults to None.

WHAT IT SHOULD DO

Set up the object's data. If the caller passes a dictionary, store it as the attendance records; if nothing is passed, start with an empty dictionary.

Returns: None

Method 2 · add_attendance**SIGNATURE**

```
def add_attendance(self, employee, status)
```

PARAMETERS

Parameter	Type	Description
employee	string	Employee name.
status	string	"P" or "A".

WHAT IT SHOULD DO

Record one day's status for an employee. If the employee has no record yet, create one; then append the new status to that employee's list.

Returns: None

SAMPLE INPUT

```
system.add_attendance("Ravi", "P")
```

SAMPLE OUTPUT

```
{
    "Ravi": ["P"]
}
```

Method 3 · perfect_attendance**SIGNATURE**

```
def perfect_attendance(self)
```

WHAT IT SHOULD DO

Return the names of all employees who have never been marked absent (their list contains no "A").

Returns: list of names

SAMPLE INPUT

```
{
    "Ravi": ["P", "P", "A"],
```

```

    "Meera": ["P", "P", "P"],
    "Karan": ["P", "P"]
}

```

SAMPLE OUTPUT

```
["Meera", "Karan"]
```

Method 4 · absent_count**SIGNATURE**

```
def absent_count(self, employee)
```

PARAMETERS

Parameter	Type	Description
employee	string	Employee name to check.

WHAT IT SHOULD DO

Count how many times the given employee was marked “A”.

Returns: integer

SAMPLE INPUT

```

employee = "Ravi"
{ "Ravi": ["P", "A", "A", "P"] }

```

SAMPLE OUTPUT

```
2
```

Method 5 · attendance_percentage**SIGNATURE**

```
def attendance_percentage(self)
```

WHAT IT SHOULD DO

For every employee, calculate the share of present days as a percentage using $(\text{present days} \div \text{total days}) \times 100$, rounded to 2 decimal places. Return one entry per employee.

Returns: dictionary { employee_name: percentage }

SAMPLE INPUT

```

{
    "Ravi": ["P", "P", "A"],
    "Meera": ["P", "P", "P"]
}

```

SAMPLE OUTPUT

```
{  
    "Ravi": 66.67,  
    "Meera": 100.0  
}
```

Method 6 · highest_attendance_employee

SIGNATURE

```
def highest_attendance_employee(self)
```

WHAT IT SHOULD DO

Return the name of the employee with the highest attendance percentage. If several employees are tied for the highest, return the first one encountered.

Returns: string (employee name)

SAMPLE INPUT

```
{  
    "Ravi": ["P", "P", "A"],  
    "Meera": ["P", "P", "P"],  
    "Karan": ["P", "P"]  
}
```

SAMPLE OUTPUT

```
"Meera"
```

Sample Data

```
attendance = {  
    "Ravi": ["P", "P", "A", "P"],  
    "Meera": ["P", "P", "P", "P"],  
    "Karan": ["P", "A", "P", "P"],  
    "Sneha": ["P", "P", "P", "A"]  
}
```

Expected Output

```
Perfect Attendance Employees:  
['Meera']
```

```
Ravi Absences:  
1
```

```
Attendance Percentage:  
{  
    'Ravi': 75.0,
```

```
'Meera': 100.0,  
'Karan': 75.0,  
'Sneha': 75.0  
}
```

Highest Attendance Employee:
Meera

Constraints

- The number of employees will not exceed 5,000.
- Each employee may have up to 365 attendance entries.
- A status is always “P” or “A”.
- Employee names are case-sensitive.
- Two identical names refer to the same employee.

Concepts Practised

- Classes and objects
- Constructor (`__init__`)
- Dictionary of lists
- Searching and counting
- Aggregation and percentage calculation
- Iterating over dictionaries
- Conditional logic and list operations
- Basic reporting-system design

Question 2 — Inventory Management System

Domain · Retail / Inventory

Problem Statement

A stationery store keeps hundreds of products such as pens, notebooks, markers and files. The manager wants a system that tracks the quantity and price of each product and reports on stock levels and inventory value.

Inventory is currently maintained by hand, so spotting low-stock items and totalling inventory value is difficult. A simple inventory-management system is required.

Objective

Design and implement a Python class named `InventoryManagementSystem` that stores product information and generates inventory reports, using dictionaries and basic OOP concepts.

Class Structure

CLASS NAME

```
class InventoryManagementSystem
```

PUBLIC ATTRIBUTE

`inventory` — dictionary (nested)

Stores product details. The key is the product name; the value is another dictionary holding that product's quantity and price.

```
{
    "Pen":      {"quantity": 50, "price": 10},
    "Notebook": {"quantity": 20, "price": 50}
}
```

Methods

Method 1 · `__init__`

SIGNATURE

```
def __init__(self, inventory=None)
```

PARAMETERS

Parameter	Type	Description
<code>inventory</code>	dict (optional)	Existing inventory. Defaults to None.

WHAT IT SHOULD DO

If a dictionary is supplied, store it as the inventory; otherwise start with an empty dictionary.

Returns: None

Method 2 · add_product

SIGNATURE

```
def add_product(self, product, quantity, price)
```

PARAMETERS

Parameter	Type	Description
product	str	Product name.
quantity	int	Available quantity.
price	int / float	Price per unit.

WHAT IT SHOULD DO

Add a new product to the inventory. If the product already exists, replace its quantity and price with the new values.

Returns: None

SAMPLE INPUT

```
system.add_product("Pen", 50, 10)
```

SAMPLE OUTPUT

```
{
    "Pen": {"quantity": 50, "price": 10}
}
```

Method 3 · update_stock

SIGNATURE

```
def update_stock(self, product, quantity)
```

PARAMETERS

Parameter	Type	Description
product	str	Existing product name.
quantity	int	Amount to add to current stock.

WHAT IT SHOULD DO

Increase the product's existing quantity by the given amount. Note this ADDS to the current stock rather than overwriting it.

Returns: None

SAMPLE INPUT

```
system.update_stock("Pen", 20)    # Pen currently has 50
```

SAMPLE OUTPUT

```
{
  "Pen": {"quantity": 70, "price": 10}
}
```

Method 4 · inventory_value**SIGNATURE**

```
def inventory_value(self)
```

WHAT IT SHOULD DO

For every product compute quantity × price, then return the sum across all products — the total money tied up in stock.

Returns: int or float

SAMPLE INPUT

```
{
  "Pen":      {"quantity": 50, "price": 10},
  "Notebook": {"quantity": 20, "price": 50}
}
```

SAMPLE OUTPUT

```
1500      # (50×10) + (20×50) = 500 + 1000
```

Method 5 · low_stock_products**SIGNATURE**

```
def low_stock_products(self, threshold)
```

PARAMETERS

Parameter	Type	Description
threshold	int	Stock cut-off level.

WHAT IT SHOULD DO

Return the names of all products whose quantity is strictly less than the threshold.

Returns: list

SAMPLE INPUT

```
threshold = 25
{
  "Pen":      {"quantity": 50, "price": 10},
  "Notebook": {"quantity": 20, "price": 50}
}
```

SAMPLE OUTPUT

```
["Notebook"]
```

Method 6 · most_stocked_product

SIGNATURE

```
def most_stocked_product(self)
```

WHAT IT SHOULD DO

Return the name of the product that has the highest quantity in stock.

Returns: string

SAMPLE INPUT

```
{
    "Pen":      {"quantity": 50, "price": 10},
    "Notebook": {"quantity": 20, "price": 50},
    "Marker":   {"quantity": 80, "price": 30}
}
```

SAMPLE OUTPUT

```
"Marker"
```

Method 7 · total_products

SIGNATURE

```
def total_products(self)
```

WHAT IT SHOULD DO

Return how many unique products are in the inventory.

Returns: int

SAMPLE INPUT

```
{ "Pen": {...}, "Notebook": {...}, "Marker": {...} }
```

SAMPLE OUTPUT

```
3
```

Sample Data

```
{
    "Pen":      {"quantity": 50, "price": 10},
    "Notebook": {"quantity": 20, "price": 50},
    "Marker":   {"quantity": 80, "price": 30},
    "File":     {"quantity": 15, "price": 40}
}
```

```
}
```

Expected Output

```
Total Inventory Value:  
4500
```

```
Low Stock Products (< 25):  
['Notebook', 'File']
```

```
Most Stocked Product:  
Marker
```

```
Total Products:  
4
```

Constraints

- The number of products will not exceed 5,000.
- Quantity is always non-negative.
- Price is always positive.
- Product names are case-sensitive.
- Duplicate product names are not allowed.

Concepts Practised

- Classes and objects
- Constructor
- Nested dictionaries and traversal
- Aggregation and filtering
- Searching with `max()` and a key
- Inventory reporting and business logic

Question 3 — Hospital Appointment System

Domain · Healthcare

Problem Statement

A hospital sees hundreds of patients each day across many doctors. Every appointment records the patient name, the doctor name, and the appointment date.

Management wants to answer questions such as: how many appointments has each doctor handled, how many has each patient booked, which patients have seen more than one doctor, who is the busiest doctor, and how many appointments fall on a given date.

Records are kept manually today, so a simple appointment-management system is needed.

Objective

Design and implement a Python class named `HospitalAppointmentSystem` that stores appointment records and generates reports, using lists, tuples, sets and dictionaries.

Class Structure

CLASS NAME

```
class HospitalAppointmentSystem
```

PUBLIC ATTRIBUTE

`appointments` — list of tuples

Stores every appointment. Each record is a tuple of (patient_name, doctor_name, appointment_date), where the date is a string in YYYY-MM-DD format. It must be created inside the constructor.

```
[
    ("Ravi", "Dr. Shah", "2025-01-10"),
    ("Meera", "Dr. Patel", "2025-01-11"),
    ("Karan", "Dr. Shah", "2025-01-12")
]
```

Methods

Method 1 · `__init__`

SIGNATURE

```
def __init__(self, appointments=None)
```

PARAMETERS

Parameter	Type	Description
appointments	list (optional)	Existing records. Defaults to None.

WHAT IT SHOULD DO

If a list is supplied, store it as the appointments; otherwise start with an empty list.

Returns: None

Method 2 · add_appointment**SIGNATURE**

```
def add_appointment(self, patient, doctor, date)
```

PARAMETERS

Parameter	Type	Description
patient	string	Patient name.
doctor	string	Doctor name.
date	string	Date (YYYY-MM-DD).

WHAT IT SHOULD DO

Add a new appointment to the list, always stored as the tuple (patient, doctor, date).

Returns: None

SAMPLE INPUT

```
system.add_appointment("Ravi", "Dr. Shah", "2025-01-10")
```

SAMPLE OUTPUT

```
[ ("Ravi", "Dr. Shah", "2025-01-10") ]
```

Method 3 · doctor_workload**SIGNATURE**

```
def doctor_workload(self)
```

WHAT IT SHOULD DO

Count how many appointments each doctor has, returning one entry per doctor.

Returns: dictionary { doctor_name: count }

SAMPLE INPUT

```
[
    ("Ravi", "Dr. Shah", "2025-01-10"),
    ("Meera", "Dr. Shah", "2025-01-11"),
    ("Karan", "Dr. Patel", "2025-01-12")
]
```

SAMPLE OUTPUT

```
{ "Dr. Shah": 2, "Dr. Patel": 1 }
```

Method 4 · patient_visit_count

SIGNATURE

```
def patient_visit_count(self)
```

WHAT IT SHOULD DO

Count how many appointments each patient has booked.

Returns: dictionary { patient_name: count }

SAMPLE INPUT

```
[
    ("Ravi", "Dr. Shah", "2025-01-10"),
    ("Ravi", "Dr. Patel", "2025-01-15"),
    ("Meera", "Dr. Shah", "2025-01-11")
]
```

SAMPLE OUTPUT

```
{ "Ravi": 2, "Meera": 1 }
```

Method 5 · multi_doctor_patients

SIGNATURE

```
def multi_doctor_patients(self)
```

WHAT IT SHOULD DO

Find every patient who has consulted more than one distinct doctor, and return their names as a set.

Returns: set of patient names

SAMPLE INPUT

```
[
    ("Ravi", "Dr. Shah", "2025-01-10"),
    ("Ravi", "Dr. Patel", "2025-01-12"),
    ("Meera", "Dr. Shah", "2025-01-11")
]
```

SAMPLE OUTPUT

```
{ "Ravi" }
```

Method 6 · busiest_doctor

SIGNATURE

```
def busiest_doctor(self)
```

WHAT IT SHOULD DO

Return the doctor with the most appointments. If two or more tie, return the first one encountered.

Returns: string (doctor name)

SAMPLE INPUT

```
[
    ("Ravi", "Dr. Shah", "2025-01-10"),
    ("Meera", "Dr. Shah", "2025-01-11"),
    ("Karan", "Dr. Patel", "2025-01-12")
]
```

SAMPLE OUTPUT

```
"Dr. Shah"
```

Method 7 · appointments_on_date

SIGNATURE

```
def appointments_on_date(self, date)
```

PARAMETERS

Parameter	Type	Description
date	string	Date to filter by (YYYY-MM-DD).

WHAT IT SHOULD DO

Return every appointment tuple whose date matches the given date.

Returns: list of tuples

SAMPLE INPUT

```
date = "2025-01-10"
[
    ("Ravi", "Dr. Shah", "2025-01-10"),
    ("Meera", "Dr. Patel", "2025-01-10"),
    ("Karan", "Dr. Shah", "2025-01-12")
]
```

SAMPLE OUTPUT

```
[
    ("Ravi", "Dr. Shah", "2025-01-10"),
    ("Meera", "Dr. Patel", "2025-01-10")
]
```

Sample Data

```
appointments = [
    ("Ravi", "Dr. Shah", "2025-01-10"),
    ("Ravi", "Dr. Patel", "2025-01-12"),
```

```
( "Meera", "Dr. Shah", "2025-01-11"),  
( "Karan", "Dr. Shah", "2025-01-12"),  
( "Sneha", "Dr. Patel", "2025-01-10"),  
( "Karan", "Dr. Gupta", "2025-01-15")  
]
```

Expected Output

```
Doctor Workload:  
{ 'Dr. Shah': 3, 'Dr. Patel': 2, 'Dr. Gupta': 1}
```

```
Patient Visit Count:  
{ 'Ravi': 2, 'Meera': 1, 'Karan': 2, 'Sneha': 1}
```

```
Multi Doctor Patients:  
{ 'Ravi', 'Karan' }
```

```
Busiest Doctor:  
Dr. Shah
```

```
Appointments On Date 2025-01-10:  
[  
  ( 'Ravi', 'Dr. Shah', '2025-01-10'),  
  ( 'Sneha', 'Dr. Patel', '2025-01-10')  
]
```

Constraints

- Total appointments will not exceed 20,000.
- Patient names are case-sensitive.
- Doctor names are case-sensitive.
- A patient may have multiple appointments.
- Dates are always in YYYY-MM-DD format.

Concepts Practised

- Classes and objects
- Constructor
- List of tuples
- Dictionary frequency counting
- Set operations
- Searching and filtering
- Max-value identification
- Data analysis with OOP

Question 4 — Digital Wallet System

Domain · FinTech

Problem Statement

A fintech company runs a digital wallet that lets users deposit, withdraw and transfer money between wallets.

The company wants a system that keeps wallet balances and a transaction history, and reports the current balance, total deposits, total withdrawals, the highest-balance holder, and a user's transaction history.

Transaction data is currently scattered across systems, so a single wallet-management system is required.

Objective

Design and implement a Python class named **DigitalWalletSystem** that manages wallet accounts and transactions, using dictionaries, lists and OOP concepts.

Class Structure

CLASS NAME

```
class DigitalWalletSystem
```

PUBLIC ATTRIBUTES

wallets — dictionary

Maps each user name to their current balance.

```
{
    "Ravi": 5000,
    "Meera": 8000
}
```

transactions — dictionary of lists

Maps each user to a list of their transactions, where each transaction is a tuple of (type, amount).

```
{
    "Ravi": [
        ("Deposit", 5000),
        ("Withdraw", 1000)
    ]
}
```

Methods

Method 1 · `__init__`

SIGNATURE

```
def __init__(self)
```

WHAT IT SHOULD DO

Initialise both wallets and transactions as empty dictionaries.

Returns: None

Method 2 · `create_wallet`

SIGNATURE

```
def create_wallet(self, user)
```

PARAMETERS

Parameter	Type	Description
user	str	New user name.

WHAT IT SHOULD DO

Create a wallet for the user with a starting balance of 0 (and an empty transaction history).

SAMPLE INPUT

```
system.create_wallet("Ravi")
```

SAMPLE OUTPUT

```
{ "Ravi": 0 }
```

Method 3 · `deposit`

SIGNATURE

```
def deposit(self, user, amount)
```

PARAMETERS

Parameter	Type	Description
user	str	Wallet owner.
amount	int	Amount to add.

WHAT IT SHOULD DO

Add the amount to the user's balance and record the transaction ("Deposit", amount).

SAMPLE INPUT

```
system.deposit("Ravi", 5000)
```

SAMPLE OUTPUT

```
Balance: 5000
History: [("Deposit", 5000)]
```

Method 4 · withdraw

SIGNATURE

```
def withdraw(self, user, amount)
```

PARAMETERS

Parameter	Type	Description
user	str	Wallet owner.
amount	int	Amount to remove.

WHAT IT SHOULD DO

Subtract the amount from the user's balance, but only if the balance is sufficient (the balance must never go negative). Record the transaction ("Withdraw", amount).

SAMPLE INPUT

```
system.withdraw("Ravi", 1000)
```

SAMPLE OUTPUT

```
Balance: 4000
```

Method 5 · transfer

SIGNATURE

```
def transfer(self, sender, receiver, amount)
```

PARAMETERS

Parameter	Type	Description
sender	str	User sending money.
receiver	str	User receiving money.
amount	int	Amount to move.

WHAT IT SHOULD DO

Move money from the sender to the receiver, only if the sender has enough balance. Record ("Transfer Out", amount) for the sender and ("Transfer In", amount) for the receiver.

SAMPLE INPUT

```
system.transfer("Ravi", "Meera", 2000)
```

SAMPLE OUTPUT

```
Ravi  -> 2000
Meera -> 10000
```

Method 6 · current_balance

SIGNATURE

```
def current_balance(self, user)
```

WHAT IT SHOULD DO

Return the user's current wallet balance.

Returns: int

SAMPLE OUTPUT

```
4000
```

Method 7 · transaction_history

SIGNATURE

```
def transaction_history(self, user)
```

WHAT IT SHOULD DO

Return the user's full list of transactions.

Returns: list

SAMPLE OUTPUT

```
[
    ("Deposit", 5000),
    ("Withdraw", 1000),
    ("Transfer Out", 2000)
]
```

Method 8 · total_deposits

SIGNATURE

```
def total_deposits(self)
```

WHAT IT SHOULD DO

Add up the amounts of every "Deposit" transaction across all users.

Returns: int

Read this carefully

Only "Deposit" actions count here — "Transfer In" and "Transfer Out" are not deposits.

SAMPLE OUTPUT

```
13000
```

Method 9 · highest_balance_holder

SIGNATURE

```
def highest_balance_holder(self)
```

WHAT IT SHOULD DO

Return the user who currently has the largest balance.

Returns: str

SAMPLE OUTPUT

```
"Meera"
```

Sample Data

```
Ravi
  Deposit: 5000
  Withdraw: 1000
  Transfer: 2000 to Meera

Meera
  Deposit: 8000
  Receive: 2000 from Ravi
```

Expected Output

```
Current Balances:
{'Ravi': 2000, 'Meera': 10000}

Highest Balance Holder:
Meera

Total Deposits:
13000      # 5000 (Ravi) + 8000 (Meera)

Transaction History (Ravi):
[
  ('Deposit', 5000),
  ('Withdraw', 1000),
  ('Transfer Out', 2000)
]
```

Constraints

- Number of wallet users $\leq 10,000$.

- A balance can never become negative.
- A transfer is allowed only if the sender has enough balance.
- Amounts are positive integers.
- User names are unique.

Concepts Practised

- Classes and objects
- Dictionary of lists and nested structures
- Transaction processing and validation
- Aggregation and searching
- Financial calculations
- Real-world FinTech design

Question 5 — Online Shopping Cart System

Domain · E-Commerce

Problem Statement

An e-commerce company lets customers add products to a shopping cart before placing an order.

The company wants a system that manages carts and reports the products in a customer's cart, the total cart value, the most expensive product, the total number of items, which customers have active carts, and how popular each product is across all carts.

Cart data is currently temporary and reports are produced by hand, so a simple cart-management system is needed.

Objective

Design and implement a Python class named **OnlineShoppingCartSystem** that manages customer carts and generates cart reports, using dictionaries, lists and OOP concepts.

Class Structure

CLASS NAME

```
class OnlineShoppingCartSystem
```

PUBLIC ATTRIBUTE

carts — dictionary (nested)

Maps each customer to their cart. A cart maps each product name to a dictionary holding that product's price and quantity. Created inside the constructor.

```
{
    "Ravi": {
        "Laptop": {"price": 50000, "quantity": 1},
        "Mouse": {"price": 500, "quantity": 2}
    }
}
```

Methods

Method 1 · `__init__`

SIGNATURE

```
def __init__(self, carts=None)
```

PARAMETERS

Parameter	Type	Description
carts	dict (optional)	Existing carts. Defaults to None.

WHAT IT SHOULD DO

Store the supplied carts dictionary, or start with an empty dictionary if none is given.

Returns: None

Method 2 · add_to_cart**SIGNATURE**

```
def add_to_cart(self, customer, product, price, quantity)
```

PARAMETERS

Parameter	Type	Description
customer	str	Customer name.
product	str	Product name.
price	int / float	Unit price.
quantity	int	Quantity to add.

WHAT IT SHOULD DO

Add a product to a customer's cart. If the customer has no cart yet, create one. If the product is already in the cart, increase its quantity by the given amount; otherwise add it as a new product.

Returns: None

SAMPLE INPUT

```
system.add_to_cart("Ravi", "Laptop", 50000, 1)
```

SAMPLE OUTPUT

```
{
  "Ravi": {
    "Laptop": {"price": 50000, "quantity": 1}
  }
}
```

Method 3 · remove_from_cart**SIGNATURE**

```
def remove_from_cart(self, customer, product)
```

PARAMETERS

Parameter	Type	Description
customer	str	Customer name.
product	str	Product to remove.

WHAT IT SHOULD DO

Remove the named product from the customer's cart.

SAMPLE INPUT

```
system.remove_from_cart("Ravi", "Mouse")
```

SAMPLE OUTPUT

```
Mouse removed from Ravi's cart
```

Method 4 · cart_items**SIGNATURE**

```
def cart_items(self, customer)
```

WHAT IT SHOULD DO

Return the list of product names currently in the customer's cart.

Returns: list

SAMPLE OUTPUT

```
["Laptop", "Mouse", "Keyboard"]
```

Method 5 · total_cart_value**SIGNATURE**

```
def total_cart_value(self, customer)
```

WHAT IT SHOULD DO

For each product in the cart compute price × quantity, then return the total.

Returns: int or float

SAMPLE INPUT

```
Laptop: 50000 × 1  
Mouse:   500 × 2
```

SAMPLE OUTPUT

```
51000
```

Method 6 · total_items**SIGNATURE**

```
def total_items(self, customer)
```

WHAT IT SHOULD DO

Return the total quantity of items in the cart (the sum of all product quantities, not the number of distinct products).

Returns: int

SAMPLE INPUT

```
Laptop -> 1  
Mouse  -> 2
```

SAMPLE OUTPUT

```
3
```

Method 7 · most_expensive_product

SIGNATURE

```
def most_expensive_product(self, customer)
```

WHAT IT SHOULD DO

Return the product with the highest unit price in the customer's cart.

Returns: string

Note

This compares the per-unit price, not the line total (price × quantity).

SAMPLE OUTPUT

```
"Laptop"
```

Method 8 · active_customers

SIGNATURE

```
def active_customers(self)
```

WHAT IT SHOULD DO

Return the customers who have at least one item in their cart.

Returns: list

SAMPLE OUTPUT

```
["Ravi", "Meera"]
```

Method 9 · product_popularity

SIGNATURE

```
def product_popularity(self)
```

WHAT IT SHOULD DO

Across every customer's cart, add up the quantity of each product to see how popular it is overall.

Returns: dictionary { product_name: total_quantity }

SAMPLE INPUT

```
Ravi: Laptop(1), Mouse(2)
Meera: Mouse(1), Keyboard(1)
```

SAMPLE OUTPUT

```
{
    "Laptop": 1,
    "Mouse": 3,
    "Keyboard": 1
}
```

Sample Data

```
Ravi
  Laptop  50000  Qty: 1
  Mouse   500   Qty: 2

Meera
  Keyboard 1500   Qty: 1
  Mouse    500   Qty: 1

Karan
  Monitor 12000   Qty: 1
  Mouse   500    Qty: 3
```

Expected Output

```
Cart Value (Ravi):
51000

Total Items (Ravi):
3

Most Expensive Product (Ravi):
Laptop

Active Customers:
['Ravi', 'Meera', 'Karan']

Product Popularity:
{
    'Laptop': 1,
    'Mouse': 6,
```

```
'Keyboard': 1,  
'Monitor': 1  
}
```

Constraints

- Number of customers $\leq 10,000$.
- Quantity is always positive.
- Price is always positive.
- Product names are case-sensitive.
- Customer names are case-sensitive.

Concepts Practised

- Classes and objects
- Nested dictionaries
- CRUD operations (add / remove)
- Aggregation and product analytics
- Searching with `max()` and a key
- Dictionary traversal
- E-commerce domain modelling

Question 6 — Movie Streaming Platform

Domain · Entertainment

Problem Statement

A movie streaming company offers thousands of titles. Users can watch movies, rate them, and build up a watch history.

The company wants a system that tracks viewing activity and reports the movies a user has watched, the number of views per movie, a movie's average rating, the most-watched movie, the top-rated movie, and a user's total watch time.

Activity data is currently spread across systems, so a single analytics system is required.

Objective

Design and implement a Python class named `MovieStreamingPlatform` that manages viewing records and generates analytics, using dictionaries, lists, tuples and OOP concepts.

Class Structure

CLASS NAME

```
class MovieStreamingPlatform
```

PUBLIC ATTRIBUTES

`watch_history` — dictionary of lists

Maps each user to a list of the movies they watched, where each entry is a tuple of (movie_name, watch_time_minutes).

```
{
    "Ravi": [
        ("Inception", 120),
        ("Avatar", 150)
    ]
}
```

`ratings` — dictionary of lists

Maps each movie to the list of ratings it has received.

```
{
    "Inception": [5, 4, 5],
    "Avatar": [4, 5]
}
```

Methods

Method 1 · `__init__`

SIGNATURE

```
def __init__(self)
```

WHAT IT SHOULD DO

Initialise `watch_history` and `ratings` as empty dictionaries.

Returns: None

Method 2 · `watch_movie`

SIGNATURE

```
def watch_movie(self, user, movie, duration)
```

PARAMETERS

Parameter	Type	Description
<code>user</code>	<code>str</code>	Viewer name.
<code>movie</code>	<code>str</code>	Movie title.
<code>duration</code>	<code>int</code>	Minutes watched.

WHAT IT SHOULD DO

Record that a user watched a movie by appending `(movie, duration)` to that user's watch history (creating the user's list if needed).

Returns: None

SAMPLE INPUT

```
system.watch_movie("Ravi", "Inception", 120)
```

SAMPLE OUTPUT

```
{ "Ravi": [("Inception", 120)] }
```

Method 3 · `rate_movie`

SIGNATURE

```
def rate_movie(self, movie, rating)
```

PARAMETERS

Parameter	Type	Description
<code>movie</code>	<code>str</code>	Movie title.
<code>rating</code>	<code>int</code>	Rating from 1 to 5.

WHAT IT SHOULD DO

Add the rating to that movie's list of ratings (creating the list if needed).

Returns: None

SAMPLE INPUT

```
system.rate_movie("Inception", 5)
```

SAMPLE OUTPUT

```
{ "Inception": [5] }
```

Method 4 · movies_watched_by_user

SIGNATURE

```
def movies_watched_by_user(self, user)
```

WHAT IT SHOULD DO

Return the list of movie titles the given user has watched.

Returns: list

SAMPLE OUTPUT

```
["Inception", "Avatar"]
```

Method 5 · total_watch_time

SIGNATURE

```
def total_watch_time(self, user)
```

WHAT IT SHOULD DO

Add up all the watch durations in the user's history and return the total in minutes.

Returns: int

SAMPLE INPUT

```
[("Inception", 120), ("Avatar", 150)]
```

SAMPLE OUTPUT

```
270
```

Method 6 · movie_view_count

SIGNATURE

```
def movie_view_count(self)
```

WHAT IT SHOULD DO

Count how many times each movie was watched across all users.

Returns: dictionary { movie_name: view_count }

SAMPLE INPUT

```
Ravi: Inception  
Meera: Inception  
Karan: Avatar
```

SAMPLE OUTPUT

```
{ "Inception": 2, "Avatar": 1 }
```

Method 7 · average_rating**SIGNATURE**

```
def average_rating(self, movie)
```

WHAT IT SHOULD DO

Return the average of the movie's ratings, rounded to 2 decimal places.

Returns: float

SAMPLE INPUT

```
"Inception": [5, 4, 5]
```

SAMPLE OUTPUT

```
4.67
```

Method 8 · most_watched_movie**SIGNATURE**

```
def most_watched_movie(self)
```

WHAT IT SHOULD DO

Return the movie with the highest view count.

Returns: string

SAMPLE OUTPUT

```
"Inception"
```

Method 9 · topRated_movie**SIGNATURE**

```
def topRated_movie(self)
```

WHAT IT SHOULD DO

Return the movie with the highest average rating.

Returns: string

SAMPLE OUTPUT

```
"Inception"
```

Sample Data

```
Ravi watched: Inception (120), Avatar (150)
Meera watched: Inception (120)
Karan watched: Avatar (150), Interstellar (170)
```

```
Ratings:
  Inception: 5, 4, 5
  Avatar:    4, 4
  Interstellar: 5, 5, 4
```

Expected Output

```
Movies Watched By Ravi:
['Inception', 'Avatar']

Total Watch Time (Ravi):
270

Movie View Count:
{'Inception': 2, 'Avatar': 2, 'Interstellar': 1}

Average Rating (Inception):
4.67

Most Watched Movie:
Inception

Top Rated Movie:
Interstellar
```

Heads-up: two ties in this dataset

Inception and Avatar are both watched twice, so “most watched” is a tie — returning the first one encountered (Inception) is acceptable. Likewise Inception and Interstellar both average 4.67, so “top rated” is also a tie. The expected output lists Interstellar, but either tied title is a defensible answer unless your tie-breaking rule says otherwise.

Constraints

- Number of users $\leq 100,000$.
- Ratings range from 1 to 5.
- Watch duration is positive.

- Movie names are case-sensitive.
- A user can watch the same movie multiple times.

Concepts Practised

- Classes and objects
- Dictionary of lists and list of tuples
- Analytics and reporting
- Average calculation and frequency counting
- Nested iteration
- Entertainment / streaming domain modelling

Question 7 — Gaming Tournament Management System

Domain · Gaming

Problem Statement

An online gaming platform runs tournaments for games such as Valorant, BGMI, Free Fire and FIFA. Players earn points based on how they perform.

The platform wants a system that manages participation records and reports each player's total points, how many tournaments they played, the top performer, a leaderboard, players active in multiple games, and a player's average points.

Results are stored manually today, so a simple tournament-management system is needed.

Objective

Design and implement a Python class named `GamingTournamentSystem` that stores tournament records and generates performance reports, using dictionaries, lists, tuples, sets and OOP concepts.

Class Structure

CLASS NAME

```
class GamingTournamentSystem
```

PUBLIC ATTRIBUTE

`tournament_records` — dictionary of lists

Maps each player to a list of their results, where each result is a tuple of (game_name, points). Created inside the constructor.

```
{
    "Ravi": [
        ("Valorant", 100),
        ("BGMI", 80)
    ],
    "Meera": [
        ("Valorant", 120)
    ]
}
```

Methods

Method 1 · `__init__`

SIGNATURE

```
def __init__(self)
```

WHAT IT SHOULD DO

Initialise `tournament_records` as an empty dictionary.

Returns: None

Method 2 · `add_result`

SIGNATURE

```
def add_result(self, player, game, points)
```

PARAMETERS

Parameter	Type	Description
player	str	Player name.
game	str	Game name.
points	int	Points scored.

WHAT IT SHOULD DO

Add a tournament result by appending (game, points) to the player's record (creating it if needed).

Returns: None

SAMPLE INPUT

```
system.add_result("Ravi", "Valorant", 100)
```

SAMPLE OUTPUT

```
{ "Ravi": [("Valorant", 100)] }
```

Method 3 · `tournaments_played`

SIGNATURE

```
def tournaments_played(self, player)
```

WHAT IT SHOULD DO

Return how many tournaments the player has taken part in (the number of entries in their record).

Returns: int

SAMPLE INPUT

```
Ravi: [("Valorant", 100), ("BGMI", 80)]
```

SAMPLE OUTPUT

```
2
```

Method 4 · `total_points`

SIGNATURE

```
def total_points(self, player)
```

WHAT IT SHOULD DO

Add up all the points the player has earned.

Returns: int

SAMPLE INPUT

```
[("Valorant", 100), ("BGMI", 80)]
```

SAMPLE OUTPUT

```
180
```

Method 5 · average_points**SIGNATURE**

```
def average_points(self, player)
```

WHAT IT SHOULD DO

Return the player's average points across all their tournaments, rounded to 2 decimal places.

Returns: float

SAMPLE INPUT

```
(100 + 80) / 2 tournaments
```

SAMPLE OUTPUT

```
90.0
```

Method 6 · leaderboard**SIGNATURE**

```
def leaderboard(self)
```

WHAT IT SHOULD DO

Build a leaderboard of (player, total_points) entries, sorted by total points from highest to lowest.

Returns: list of tuples

SAMPLE INPUT

```
Ravi -> 180, Meera -> 250, Karan -> 150
```

SAMPLE OUTPUT

```
[
    ("Meera", 250),
    ("Ravi", 180),
    ("Karan", 150)
```

```
]
```

Method 7 · top_player

SIGNATURE

```
def top_player(self)
```

WHAT IT SHOULD DO

Return the player with the highest total points.

Returns: string

SAMPLE OUTPUT

```
"Meera"
```

Method 8 · players_in_multiple_games

SIGNATURE

```
def players_in_multiple_games(self)
```

WHAT IT SHOULD DO

Return the set of players who took part in more than one distinct game.

Returns: set

SAMPLE INPUT

```
Ravi: Valorant, BGMI  
Meera: Valorant
```

SAMPLE OUTPUT

```
{ "Ravi" }
```

Method 9 · game_popularity

SIGNATURE

```
def game_popularity(self)
```

WHAT IT SHOULD DO

Count how many tournament entries each game has across all players.

Returns: dictionary { game_name: count }

SAMPLE INPUT

```
Ravi: Valorant  
Meera: Valorant  
Karan: BGMI
```

SAMPLE OUTPUT

```
{ "Valorant": 2, "BGMI": 1 }
```

Sample Data

```
Ravi    Valorant 100, BGMI 80, FIFA 90
Meera   Valorant 120, Free Fire 110
Karan   BGMI 70, FIFA 85
Sneha   Valorant 130, BGMI 100
```

Expected Output

```
Total Points (Ravi):
270

Average Points (Ravi):
90.0

Top Player:
Ravi

Players In Multiple Games:
{'Ravi', 'Meera', 'Karan', 'Sneha'}

Leaderboard:
[
    ('Ravi', 270),
    ('Sneha', 230),
    ('Meera', 230),
    ('Karan', 155)
]

Game Popularity:
{'Valorant': 3, 'BGMI': 3, 'FIFA': 2, 'Free Fire': 1}
```

Correction to the original document

The source listed the top player as Sneha and a mis-sorted leaderboard. With this data Ravi has the highest total ($100 + 80 + 90 = 270$), so the correct top player is **Ravi**, and the leaderboard must place Ravi first. The values shown above are the corrected ones.

Constraints

- Number of players $\leq 50,000$.
- Points are non-negative integers.
- Game names are case-sensitive.

- A player can play multiple tournaments.
- Multiple entries for the same game are allowed.

Concepts Practised

- Classes and objects
- Dictionary of lists and list of tuples
- Sorting with a key
- Leaderboard generation and ranking
- Set operations
- Aggregation and gaming analytics

Question 8 — University Course Feedback System

Domain · University / Education

Problem Statement

A university collects student feedback at the end of every semester. Students submit a rating and a comment for each course they attended.

The academic department wants a system that stores feedback and reports each course's average rating, the most highly rated course, the number of submissions per course, the courses a student reviewed, students who reviewed several courses, and an overall university rating.

Feedback forms are processed by hand today, so a simple feedback-management system is required.

Objective

Design and implement a Python class named `UniversityCourseFeedbackSystem` that stores course feedback and generates analytics, using dictionaries, lists, tuples and OOP concepts.

Class Structure

CLASS NAME

```
class UniversityCourseFeedbackSystem
```

PUBLIC ATTRIBUTE

`feedback_records` — dictionary of lists

Maps each student to a list of their feedback, where each entry is a tuple of (course_name, rating, comment). Created inside the constructor.

```
{
    "Ravi": [
        ("Python", 5, "Excellent"),
        ("SQL", 4, "Good")
    ],
    "Meera": [
        ("Python", 4, "Useful")
    ]
}
```

Methods

Method 1 · `__init__`

SIGNATURE

```
def __init__(self)
```

WHAT IT SHOULD DO

Initialise `feedback_records` as an empty dictionary.

Returns: None

Method 2 · `submit_feedback`

SIGNATURE

```
def submit_feedback(self, student, course, rating, comment)
```

PARAMETERS

Parameter	Type	Description
<code>student</code>	<code>str</code>	Student name.
<code>course</code>	<code>str</code>	Course name.
<code>rating</code>	<code>int</code>	Rating from 1 to 5.
<code>comment</code>	<code>str</code>	Free-text comment.

WHAT IT SHOULD DO

Store one piece of feedback by appending (`course`, `rating`, `comment`) to the student's history (creating it if needed).

Returns: None

SAMPLE INPUT

```
system.submit_feedback("Ravi", "Python", 5, "Excellent")
```

SAMPLE OUTPUT

```
{ "Ravi": [("Python", 5, "Excellent")] }
```

Method 3 · `courses_reviewed_by_student`

SIGNATURE

```
def courses_reviewed_by_student(self, student)
```

WHAT IT SHOULD DO

Return the list of courses the given student reviewed.

Returns: list

SAMPLE OUTPUT

```
["Python", "SQL"]
```

Method 4 · `feedback_count_per_course`

SIGNATURE

```
def feedback_count_per_course(self)
```

WHAT IT SHOULD DO

Count how many feedback submissions each course received across all students.

Returns: dictionary { course_name: count }

SAMPLE OUTPUT

```
{ "Python": 3, "SQL": 2 }
```

Method 5 · average_course_rating**SIGNATURE**

```
def average_course_rating(self, course)
```

PARAMETERS

Parameter	Type	Description
course	str	Course to evaluate.

WHAT IT SHOULD DO

Return the average rating of the given course across all students, rounded to 2 decimal places.

Returns: float

SAMPLE INPUT

```
Python ratings: 5, 4, 5
```

SAMPLE OUTPUT

```
4.67
```

Method 6 · highestRated_course**SIGNATURE**

```
def highestRated_course(self)
```

WHAT IT SHOULD DO

Return the course with the highest average rating.

Returns: string

SAMPLE OUTPUT

```
"Python"
```

Method 7 · students_reviewing_multiple_courses**SIGNATURE**

```
def students_reviewing_multiple_courses(self)
```

WHAT IT SHOULD DO

Return the set of students who reviewed more than one distinct course.

Returns: set

SAMPLE OUTPUT

```
{ "Ravi", "Sneha" }
```

Method 8 · overall_university_rating**SIGNATURE**

```
def overall_university_rating(self)
```

WHAT IT SHOULD DO

Return the average of every rating submitted (across all students and all courses), rounded to 2 decimal places.

Returns: float

SAMPLE OUTPUT

```
4.43
```

Method 9 · course_comments**SIGNATURE**

```
def course_comments(self, course)
```

PARAMETERS

Parameter	Type	Description
course	str	Course to read comments for.

WHAT IT SHOULD DO

Return all the comments submitted for the given course.

Returns: list

SAMPLE OUTPUT

```
["Excellent", "Very Useful", "Good Course"]
```

Sample Data

Ravi	Python	5	"Excellent"
	SQL	4	"Good"

Meera	Python	4	"Useful"
Karan	SQL	5	"Excellent"
Sneha	Python	5	"Very Useful"
	AI	4	"Interesting"

Expected Output

```
Courses Reviewed By Ravi:  
['Python', 'SQL']
```

```
Feedback Count Per Course:  
{'Python': 3, 'SQL': 2, 'AI': 1}
```

```
Average Rating (Python):  
4.67
```

```
Highest Rated Course:  
Python
```

```
Students Reviewing Multiple Courses:  
{'Ravi', 'Sneha'}
```

```
Overall University Rating:  
4.5
```

Correction to the original document

The source first showed “SQL” as the highest-rated course, then corrected itself. Python averages 4.67 while SQL averages 4.5, so the highest-rated course is **Python**. The overall rating is the mean of all six ratings (5, 4, 4, 5, 5, 4) = 4.5.

Constraints

- Number of students $\leq 50,000$.
- Ratings range from 1 to 5.
- Course names are case-sensitive.
- Student names are case-sensitive.
- Comments are non-empty strings.

Concepts Practised

- Classes and objects
- Dictionary of lists and list of tuples
- Analytics and reporting
- Average calculations

- Set operations and filtering
- University / academic domain modelling

Question 9 — Loan Management System

Domain · FinTech

Problem Statement

A fintech company gives personal loans to customers, and each customer can take several loans over time.

The company wants a system that tracks loan records and reports the total amount a customer borrowed, their active loans, the total outstanding amount, the biggest borrower, the spread of loans by type, and the average loan amount.

Loans are currently tracked in spreadsheets, which is slow and error-prone, so a Loan Management System is required.

Objective

Design and implement a Python class named `LoanManagementSystem` that stores customer loan information and generates loan analytics, using dictionaries, lists, tuples and OOP concepts.

Class Structure

CLASS NAME

```
class LoanManagementSystem
```

PUBLIC ATTRIBUTE

`loans` — dictionary of lists

Maps each customer to a list of their loans. Each loan is a tuple of (loan_id, loan_type, amount, status), where loan_id is a unique string, loan_type is something like Home / Personal / Education / Vehicle, amount is the loan value, and status is either “Active” or “Closed”.

```
{
    "Ravi": [
        ("L101", "Personal", 50000, "Active"),
        ("L102", "Vehicle", 200000, "Closed")
    ],
    "Meera": [
        ("L103", "Education", 300000, "Active")
    ]
}
```

Methods

Method 1 · `__init__`

SIGNATURE

```
def __init__(self)
```

WHAT IT SHOULD DO

Initialise loans as an empty dictionary.

Returns: None

Method 2 · add_loan**SIGNATURE**

```
def add_loan(self, customer, loan_id, loan_type, amount, status)
```

PARAMETERS

Parameter	Type	Description
customer	str	Customer name.
loan_id	str	Unique loan ID.
loan_type	str	Type of loan.
amount	int	Loan amount.
status	str	“Active” or “Closed”.

WHAT IT SHOULD DO

Add a loan by appending the tuple (loan_id, loan_type, amount, status) to the customer’s list (creating it if needed).

SAMPLE INPUT

```
system.add_loan("Ravi", "L101", "Personal", 50000, "Active")
```

SAMPLE OUTPUT

```
{ "Ravi": [("L101", "Personal", 50000, "Active")] }
```

Method 3 · customer_total_loans**SIGNATURE**

```
def customer_total_loans(self, customer)
```

WHAT IT SHOULD DO

Add up the amounts of all the customer’s loans (both active and closed) and return the total.

Returns: int

SAMPLE INPUT

```
[
    ("L101", "Personal", 50000, "Active"),
    ("L102", "Vehicle", 200000, "Closed")
]
```

SAMPLE OUTPUT

```
250000
```

Method 4 · active_loans**SIGNATURE**

```
def active_loans(self, customer)
```

WHAT IT SHOULD DO

Return the list of the customer's loans whose status is "Active".

Returns: list

SAMPLE OUTPUT

```
[("L101", "Personal", 50000, "Active")]
```

Method 5 · total_outstanding_amount**SIGNATURE**

```
def total_outstanding_amount(self)
```

WHAT IT SHOULD DO

Add up the amounts of every "Active" loan across all customers (closed loans are excluded).

Returns: int

SAMPLE INPUT

```
Ravi: 50000 Active  
Meera: 300000 Active  
Karan: 100000 Closed
```

SAMPLE OUTPUT

```
350000
```

Method 6 · highest_borrower**SIGNATURE**

```
def highest_borrower(self)
```

WHAT IT SHOULD DO

Return the customer whose total borrowed amount is the largest.

Returns: str

SAMPLE OUTPUT

```
"Meera"
```

Method 7 · loan_type_distribution

SIGNATURE

```
def loan_type_distribution(self)
```

WHAT IT SHOULD DO

Count how many loans there are of each type across all customers.

Returns: dictionary {loan_type: count }

SAMPLE OUTPUT

```
{ "Personal": 2, "Education": 1, "Vehicle": 1 }
```

Method 8 · average_loan_amount

SIGNATURE

```
def average_loan_amount(self)
```

WHAT IT SHOULD DO

Return the average amount across all loans (active and closed), rounded to 2 decimal places.

Returns: float

SAMPLE OUTPUT

```
162500.0
```

Method 9 · customers_with_multiple_loans

SIGNATURE

```
def customers_with_multiple_loans(self)
```

WHAT IT SHOULD DO

Return the customers who have more than one loan.

Returns: list

SAMPLE OUTPUT

```
["Ravi", "Sneha"]
```

Sample Data

Ravi	L101	Personal	50000	Active
	L102	Vehicle	200000	Closed
Meera	L103	Education	300000	Active

Karan	L104	Personal	100000	Closed
Sneha	L105	Home	400000	Active
	L106	Personal	80000	Active

Expected Output

```
Customer Total Loans (Ravi):
250000

Active Loans (Sneha):
[
  ('L105', 'Home',      400000, 'Active'),
  ('L106', 'Personal',   80000, 'Active')
]

Total Outstanding Amount:
830000      # 50000 + 300000 + 400000 + 80000

Highest Borrower:
Sneha      # total borrowed = 480000

Loan Type Distribution:
{'Personal': 3, 'Vehicle': 1, 'Education': 1, 'Home': 1}

Average Loan Amount:
188333.33

Customers With Multiple Loans:
['Ravi', 'Sneha']
```

Constraints

- Number of customers $\leq 100,000$.
- Loan amount is always positive.
- Status is only “Active” or “Closed”.
- Loan IDs are unique.
- Customer names are case-sensitive.

Concepts Practised

- Classes and objects
- Dictionary of lists and list of tuples
- Filtering active records
- Aggregation and analytics
- Frequency counting and averages

- Customer-portfolio analysis
- FinTech domain modelling